MANUAL TRACING AND PERFORMANCE ANALYSIS

1. Introduction

Manual tracing is the process of executing an algorithm step by step using given input values to understand its working. In this project, manual tracing is performed for the Banker's Safety Algorithm, Deadlock Detection Algorithm, and Resource Request Validation.

The purpose is to understand how processes use resources, how resources are released, and how the operating system determines whether the system is in a safe or unsafe state.

2. Sample Input

Number of Processes: 5

Number of Resource Types: 3 (A, B, C)

Allocation Matrix

Process	A	B	C
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

Maximum Matrix

Process	A	B	C
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

Available Resources:

\[Available = [3,3,2] \]

3. Manual Tracing of Banker's Algorithm

Step 1: Calculate Need Matrix

The Need Matrix represents the remaining resources required by each process.

Formula:

\[Need = Maximum - Allocation \]

Process	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0

P3	0	1	1
P4	4	3	1

Step 2: Initialize Work

Initially:

\[Work = Available = [3,3,2] \]

The algorithm checks whether the Need of any unfinished process is less than or equal to Work.

Step 3: Execute Processes

Step	Process	Need	Work Before	Work After
1	P1	[1,2,2]	[3,3,2]	[5,3,2]
2	P3	[0,1,1]	[5,3,2]	[7,4,3]
3	P4	[4,3,1]	[7,4,3]	[7,4,5]
4	P0	[7,4,3]	[7,4,5]	[7,5,5]
5	P2	[6,0,0]	[7,5,5]	[10,5,7]

Step 4: Final Result

All five processes can complete their execution.

Safe Sequence:

\[P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2 \]
System Status: SAFE STATE

4. Manual Tracing of Deadlock Detection

For this example, the current outstanding Request Matrix is:

Process	A	B	C
P0	0	0	0
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	0	0	0

Initial Work:

\[Work = [3,3,2] \]

Each process can finish because its current outstanding request is zero. After completion, its allocated resources are released.

Step	Process	Work Before	Allocation Released	Work After
1	P0	[3,3,2]	[0,1,0]	[3,4,2]
2	P1	[3,4,2]	[2,0,0]	[5,4,2]
3	P2	[5,4,2]	[3,0,2]	[8,4,4]
4	P3	[8,4,4]	[2,1,1]	[10,5,5]
5	P4	[10,5,5]	[0,0,2]	[10,5,7]

Result: NO DEADLOCK DETECTED.

All processes can complete under the given Request Matrix.

5. Manual Tracing of Resource Request

Suppose process P1 makes an additional resource request:

$$ Request_1 = [1,0,2] $$

Step 1: Check Need

P1 Need:

$$ [1,2,2] $$

Request:

$$ [1,0,2] $$

The request does not exceed the remaining Need.

Step 2: Check Available

Available:

$$ [3,3,2] $$

The requested resources are currently available.

Step 3: Temporary Allocation

New Available:

$$ [3,3,2]-[1,0,2]=[2,3,0] $$

New Allocation of P1:

$$ [2,0,0]+[1,0,2]=[3,0,2] $$

Step 4: Run Safety Algorithm Again

The system can still complete all processes in the following sequence:

$$ P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2 $$

Request Status: GRANTED

The resource request is granted because the system remains in a safe state.

6. Performance Analysis

Performance analysis evaluates the efficiency of the algorithms in terms of execution time and memory usage.

Time and Space Complexity

Let:

- n = Number of processes
- m = Number of resource types

Operation	Time Complexity	Space Complexity
Need Matrix Calculation	O(n × m)	O(n × m)
Banker's Safety Algorithm	$O(n^2 \times m)$	O(n + m) auxiliary
Deadlock Detection	$O(n^2 \times m)$	O(n + m) auxiliary
Resource Request Validation	$O(n^2 \times m)$	O(n × m) including temporary matrix

Performance Observations

1. The program efficiently processes small and medium-sized resource allocation matrices.
2. The Need Matrix is calculated using simple subtraction operations.
3. The safety algorithm repeatedly checks whether processes can complete.
4. Deadlock detection checks outstanding resource requests.
5. Resource request validation performs another safety check after temporary allocation.

6. Memory usage depends on the number of processes and resource types.

7. Overall Analysis

The manual tracing demonstrates that the Banker's Algorithm successfully identifies a safe sequence for the given resource allocation.

The deadlock detection procedure reports no deadlock when all current outstanding requests are zero.

The resource request validation example shows that additional resources can be granted when the resulting system state remains safe.

The project demonstrates the practical implementation of deadlock avoidance, detection, and resource request validation using Python.

8. Conclusion

The Deadlock Detection and Avoidance Simulator provides a practical understanding of important operating system concepts.

Through manual tracing and performance analysis, the working of the Banker's Algorithm and deadlock detection procedure becomes easier to understand.

The project also demonstrates how Python can be used to implement resource allocation algorithms and analyze safe and unsafe system states.

Thus, the project successfully demonstrates deadlock avoidance and detection through a simple console-based Python application.